

Phishing Simulation and Encoded PowerShell Detection

Adversary Emulation and Telemetry Analysis Report

Prepared By: Jeevee Balangan, Security Analyst

Environment: Tenelex Command Center

Date: February 2026

**Platform: Wazuh SIEM | Sysmon | Windows 10 Endpoint | Ubuntu
Server 24.04**

Scope:

Controlled phishing simulation and encoded PowerShell execution to validate logging visibility, detection capability, and SIEM ingestion accuracy.

EXECUTIVE SUMMARY

This report documents a controlled phishing simulation conducted within the Tenelex Command Center lab environment. The objective was to validate that Wazuh SIEM properly ingests and records Windows Security, Sysmon, and PowerShell telemetry prior to and during execution of a simulated malicious payload.

The simulation included execution of a PowerShell script using ExecutionPolicy Bypass and an EncodedCommand argument to emulate common attacker tradecraft observed in phishing campaigns. The activity generated Sysmon Event ID 1 process creation logs and successfully exposed command-line telemetry within the Wazuh archive logs.

The lab confirms that:

- Endpoint telemetry is functioning correctly.
- Process creation events are captured with full command-line arguments.
- ExecutionPolicy Bypass and encoded PowerShell activity are visible in logs.
- The SIEM pipeline is capable of supporting detection engineering use cases.

This report captures both adversary emulation and defensive detection validation within a controlled lab environment, structured to reflect real-world SOC investigative practices.

Threat Model Context

This simulation assumes a user is successfully socially engineered into executing a malicious attachment delivered via phishing. The focus of this exercise is post-click execution visibility and detection validation rather than email gateway bypass. The goal is to evaluate endpoint telemetry and detection capability once initial access has occurred.

Lab Overview

Attack Scenario Context

This lab simulates a phishing-based initial access scenario in which a user is socially engineered into executing a malicious script delivered through a deceptive file (Invoice_Update.ps1). The objective is not to deliver real malware, but to emulate the execution behaviors commonly observed following a successful phishing attempt.

The simulation represents the following attack chain:

- User receives a phishing email containing a malicious attachment
- User executes the attachment
- PowerShell is invoked with ExecutionPolicy Bypass
- Encoded PowerShell commands are used to obscure intent
- Process creation telemetry is generated

Phishing campaigns frequently leverage PowerShell-based payload execution combined with execution policy bypass and Base64-encoded commands to evade basic detection mechanisms.

Phases 2A and 2B of this lab simulate the post-click execution behavior of such an attack in order to validate detection visibility and investigative capability within the Wazuh SIEM environment.

This validation phase served as a baseline to confirm that process creation events, command-line arguments, and execution behaviors are fully captured before simulating adversarial techniques.

Attack Flow Diagram

This simulation followed the behavioral sequence below:

1. Phishing email delivered with invoice-themed attachment

2. User executes Invoice_Update.ps1
3. PowerShell launches
4. ExecutionPolicy Bypass is used
5. EncodedCommand parameter is supplied
6. Sysmon Event ID 1 is generated
7. Wazuh agent forwards logs to manager
8. Event ingested into archives.log
9. Detection logic flags high-risk parameters
10. Analyst extracts CommandLine field
11. Base64 string decoded
12. Command intent evaluated

Phase 1 – Logging Pipeline Validation and Incident Preparation

Objective:

The objective of this lab was to validate end-to-end telemetry ingestion within the Wazuh SIEM environment prior to executing simulated phishing payload behavior. Specifically, this phase focused on confirming visibility into Windows Security logs, Sysmon process creation events, and PowerShell execution activity. Establishing reliable log ingestion ensures that subsequent attack simulations can be properly detected, analyzed, and documented.

Environment:

Wazuh Manager: Ubuntu Server 24.04

Endpoint: Windows 10 (AD01)

Agent ID: 001

Log Sources: Windows Security, Application, Sysmon, PowerShell

2.1 Agent Connectivity Verification

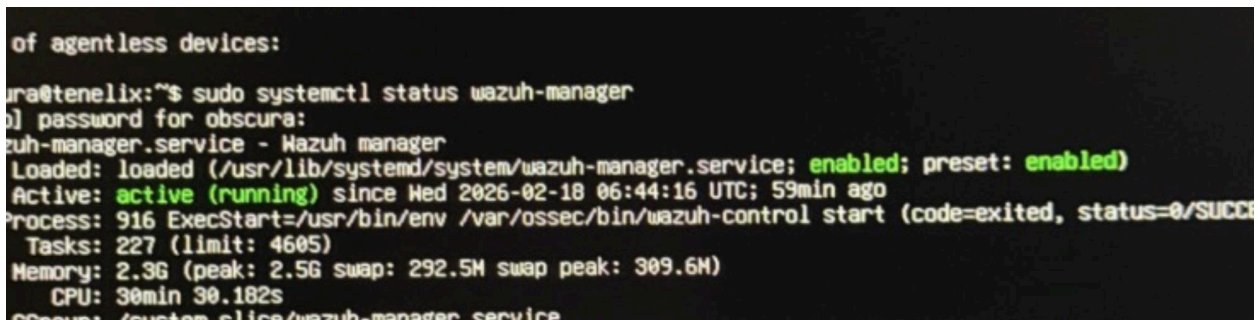
The AD01 endpoint agent was verified to be active and properly connected to the Wazuh Manager. Connectivity validation included confirming agent status and verifying that the Wazuh manager was listening on the appropriate ingestion port.

The following checks were performed:

- Confirmed AD01 agent status was “Active”
- Verified successful communication with Wazuh Manager
- Confirmed port 1514/tcp was listening for agent log ingestion

These checks ensured that endpoint telemetry would be successfully transmitted to the SIEM prior to testing.

Evidence: Initial screenshot captured during reconnection state. Subsequent verification confirmed Active status.

A terminal window showing the command 'sudo systemctl status wazuh-manager' being executed. The output indicates that the service is loaded, enabled, and active (running) since March 2026. It also shows process details, task limits, memory usage, and CPU time.

```
of agentless devices:
obscura@tenelix:~$ sudo systemctl status wazuh-manager
[sudo] password for obscura:
wazuh-manager.service - Wazuh manager
Loaded: loaded (/usr/lib/systemd/system/wazuh-manager.service; enabled; preset: enabled)
Active: active (running) since Wed 2026-02-18 06:44:16 UTC; 59min ago
Process: 916 ExecStart=/usr/bin/env /var/ossec/bin/wazuh-control start (code=exited, status=0/SUCCESS)
Tasks: 227 (limit: 4605)
Memory: 2.3G (peak: 2.5G swap: 292.5M swap peak: 309.6M)
CPU: 30min 30.182s
CGroup: /system.slice/wazuh-manager.service
```

Screenshot – (agent_control -l)

Executed on 2026-02-18 12:44 UTC

A terminal window showing two commands: 'sudo netstat -tulnp | grep 1514' which returns 'command not found', and 'sudo ss -tulnp | grep 1514' which shows a listening TCP socket on port 1514.

```
obscura@tenelix:~$ sudo netstat -tulnp | grep 1514
sudo: netstat: command not found
obscura@tenelix:~$ sudo ss -tulnp | grep 1514
tcp        LISTEN    0      128      0.0.0.0:1514      0.0.0.0:*      users:(("
```

Screenshot – (ss -tulnp | grep 1514)
Executed on 2026-02-18 01:25 UTC

2.2 Windows Event Log Ingestion Validation

To confirm Windows event ingestion functionality, a manual test event was generated using:

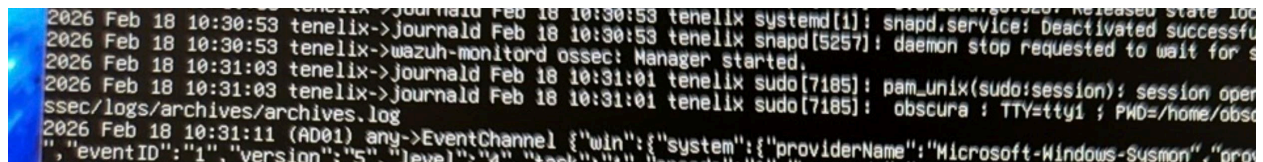
```
eventcreate /ID 1000 /L APPLICATION /T INFORMATION /SO TestSource /D "Manual test event"
```

The generated event was successfully observed within:

```
/var/ossec/logs/archives/archives.log
```

This confirmed that Windows Application logs were being properly collected, forwarded, and archived by Wazuh.

Evidence:

A screenshot of a terminal window displaying system logs. The logs show various system events, including snapd service deactivation, ossec manager startup, and a sudo session. The final line of the screenshot shows a Sysmon event entry: "eventID": "1", "version": "5", "level": "4", "task": "process_create", "providerName": "Microsoft-Windows-Sysmon", "prov".

Screenshot – (archives.log showing Application log entry)
Executed on 2026-02-18 03:30 UTC

2.3 Sysmon Process Creation Validation

To validate Sysmon process creation logging and command-line visibility, the following command was executed:

```
whoami
```

The corresponding Sysmon Event ID 1 (Process Create) entry was successfully captured in the Wazuh archives. The event included:

- Image (whoami.exe)
- Parent process details
- Full command-line arguments
- Process GUID
- User context
- Integrity level

Process creation telemetry was fully captured, including parent-child relationships and complete command-line arguments.

Evidence:



The screenshot displays a Windows Event Viewer log entry for a process creation event. The event details include the process name 'whoami.exe', its file version '10.0.19041.1 (WinBuild.160101.0800)', and the user context 'DESKTOP-2V6QGA9\AD01'. The event also shows the parent process GUID '{6fd424f7-98d4-6995-1305-000000001400}' and the parent process ID '1'. The command line is recorded as 'C:\Windows\system32\cmd.exe /c whoami'. The event source is 'Microsoft-Windows-Sysmon/Operational'.

Screenshot – (archives.log showing whoami.exe execution) Executed on 2026-02-18 03:47 UTC:

2.4 PowerShell Execution Policy Bypass Simulation

To simulate adversarial script execution behavior commonly observed in phishing and malware delivery, the following command was executed:

powershell -ExecutionPolicy Bypass -File .\Invoice_Update.ps1

This step was performed to validate detection of:

- PowerShell execution events
- ExecutionPolicy Bypass usage
- Full command-line logging
- Parent-child process relationships

The resulting logs confirmed that ExecutionPolicy Bypass usage was clearly visible in Sysmon Event ID 1 telemetry, along with associated command-line arguments and process context.

Evidence:



Screenshot – (archives.log showing ExecutionPolicy Bypass) Executed on 2026-02-18 03:52 UTC

Phase 1 Conclusion

The logging pipeline was successfully validated prior to phishing simulation activity. The following capabilities were confirmed:

- Agent connectivity between AD01 and Wazuh Manager
- Successful ingestion of Windows Application logs
- Functional Sysmon process creation logging (Event ID 1)
- Full PowerShell command-line visibility

- Detection of ExecutionPolicy Bypass execution behavior

With telemetry confirmed operational, the environment was validated as ready for advanced adversarial simulation, including encoded PowerShell command execution and phishing-based payload behavior.

MITRE ATT&CK MAPPING

This simulation aligns with the following techniques:

T1566.001 – Phishing: Spearphishing Attachment

The simulated invoice-themed payload reflects common phishing delivery methods used to initiate endpoint compromise.

T1059.001 – Command and Scripting Interpreter: PowerShell

PowerShell was used to execute commands and simulate post-delivery script execution.

T1059 – Command and Scripting Interpreter

Execution of whoami demonstrates command-line enumeration behavior typical during post-compromise discovery.

T1027 – Obfuscated/Compressed Files and Information

The use of -EncodedCommand reflects common attacker techniques to obfuscate payload content and evade simple signature-based detection.

This mapping demonstrates that the lab exercise replicates realistic adversary tradecraft rather than arbitrary command execution.

Phase 2A–2B: Encoded PowerShell Command Execution Simulation

This phase simulated adversarial use of encoded PowerShell commands to validate endpoint telemetry and detection visibility. Attackers frequently use Base64-encoded commands in combination with ExecutionPolicy bypass to evade basic logging controls and obscure intent.

The objectives of this phase were:

- Generate a Base64-encoded PowerShell command

- Execute the encoded command using ExecutionPolicy Bypass
- Validate Sysmon Event ID 1 process creation telemetry
- Confirm that encoded command strings are visible in Wazuh logs
- Decode the Base64 string to demonstrate investigative workflow

Phase 2A: Execution Policy Bypass Validation

Objective:

To confirm that PowerShell execution policy bypass attempts are logged and visible in endpoint telemetry.

Actions Performed:

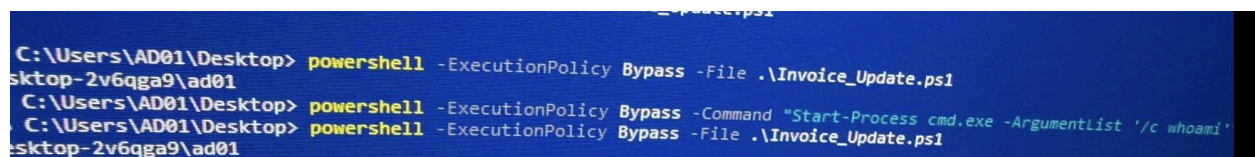
A PowerShell script was executed using:

powershell -ExecutionPolicy Bypass -File Invoice_Update.ps1

This step simulated a common attacker technique used to bypass local script execution restrictions.

Evidence:

PowerShell window showing ExecutionPolicy Bypass command.



```
C:\Users\AD01\Desktop> powershell -ExecutionPolicy Bypass -File .\Invoice_Update.ps1
sktop-2v6qga9\ad01
C:\Users\AD01\Desktop> powershell -ExecutionPolicy Bypass -Command "Start-Process cmd.exe -ArgumentList '/c whoami'
C:\Users\AD01\Desktop> powershell -ExecutionPolicy Bypass -File .\Invoice_Update.ps1
sktop-2v6qga9\ad01
```

Screenshot – (PowerShell execution of Execution Policy)

Executed on 2026-02-19 10:45 UTC

Wazuh/Sysmon log entry displaying:

Screenshot – (ExecutionPolicy Bypass)

Executed on 2026-02-19 11:05 UTC

- Event ID 1 (Process Creation)
- Image: powershell.exe
- CommandLine field containing “-ExecutionPolicy Bypass”
- Parent process information
- Process GUID

What the Reader Should Look For in the Log:

- The presence of “-ExecutionPolicy Bypass” in the CommandLine field
- Sysmon Event ID 1 indicating process creation
- The parent-child process relationship
- User context under which the command executed

Analysis:

Although the script content itself was benign, the execution method reflects real-world attacker tradecraft. ExecutionPolicy Bypass is frequently used in malware delivery and phishing-based payload execution.

Phase 2B: Encoded Command Simulation

Objective:

To simulate obfuscated PowerShell command execution using the -EncodedCommand flag and validate detection visibility.

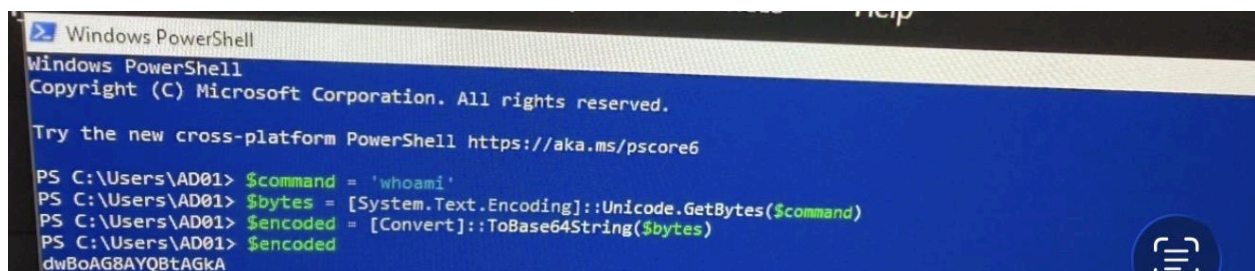
Command Encoding Process:

The following PowerShell commands were used to encode a test command:

```
$command = 'whoami'  
$bytes = [System.Text.Encoding]::Unicode.GetBytes($command)  
$encoded = [Convert]::ToBase64String($bytes)
```

Evidence:

PowerShell window showing the encoding process and generated Base64 string.



Screenshot – (Encoding Process Base64String)

Executed on 2026-02-19 11:18 UTC

Execution Step:

The encoded string was executed using:

```
powershell -ExecutionPolicy Bypass -EncodedCommand <Base64String>
```

Evidence:

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Try the new cross-platform PowerShell https://aka.ms/pscore6

PS C:\Users\AD01> $command = "whoami"
PS C:\Users\AD01> $bytes = [System.Text.Encoding]::Unicode.GetBytes($command)
PS C:\Users\AD01> $encoded = [Convert]::ToBase64String($bytes)
PS C:\Users\AD01> powershell -ExecutionPolicy Bypass -EncodedCommand $encoded
desktop-2v6qga9\ad01
PS C:\Users\AD01> powershell -ExecutionPolicy Bypass -EncodedCommand $encoded
desktop-2v6qga9\ad01
PS C:\Users\AD01>
```

Screenshot – (PowerShell window showing encoded command) Executed on 2026-02-19 11:21 UTC

Screenshot 5: Log showing ExecutionPolicy Bypass Encoded Command

Evidence:

```
{ "eventId": "1", "version": "5", "level": "4", "task": "1", "opcode": 0, "keywords": "0x8000000000000000", "systemTime": "2026-02-19T07:19:05.9337096Z", "eventRe
8871", "processId": "2876", "threadId": "1016", "channel": "Microsoft-Windows-Sysmon/Operational", "computer": "DESKTOP-2V6QGA9", "severityValue": "INFORMATION"
: "\Process Create:\r\nRuleName: \r\nUtcTime: 2026-02-19 07:19:05.990\r\nProcessGuid: {6fd424f7-b969-6996-5f02-000000001600}\r\nProcessId: 1624\r\nIm
indows\System32\whoami.exe\r\nFileVersion: 10.0.19041.1 (MinBuild.160101.0800)\r\nDescription: whoami - displays logged on user information\r\nProdu
oft\Windows\Operating System\r\nCompany: Microsoft Corporation\r\nOriginalFileName: whoami.exe\r\nCommandLine: \"C:\\Windows\\System32\\whoami.exe\"
Directory: C:\\Users\\AD01\\User: DESKTOP-2V6QGA9\\AD01\r\nLogonGuid: {6fd424f7-a15a-6996-1f8f-0b0000000000}\r\nLogonId: 0x8bf1f\r\nTerminalSessi
nIntegrityLevel: Medium\r\nHashes: MD5=A4A6924F3EAF97981323703038FD99C4, SHA256=1D4982A04D99E8CC8FE7085E63155955FEE3974490386453F6C452AE407B8743, IMPHAS
766F747CE57DFAC70743FB88\r\nParentProcessGuid: {6fd424f7-b969-6996-5e02-000000001600}\r\nParentProcessId: 4020\r\nParentImage: C:\\Windows\\System32\\
erShell\\v1.0\\powershell.exe\r\nParentCommandLine: \"C:\\Windows\\System32\\WindowsPowerShell\\v1.0\\powershell.exe\" -ExecutionPolicy Bypass -Encoded
uBoAG8AYQBtAGkA\r\nParentUser: DESKTOP-2V6QGA9\\AD01\", \"eventdata\": { \"UtcTime\": \"2026-02-19 07:19:05.990\", \"processGuid\": \"{6fd424f7-b969-6996-5f02-000000001600}\", \"processId\": \"1624\", \"image\": \"C:\\\\Windows\\\\System32\\\\whoami.exe\" } }
```

Screenshot – (ExecutionPolicy Bypass Encoded Command) Executed on 2026-02-19 11:26 UTC

Wazuh/Sysmon log showing:

- Event ID 1
- Image: powershell.exe
- CommandLine containing “-EncodedCommand”
- The Base64 string visible in telemetry

[illegible]

What the Reader Should Look For in the Log:

- Use of the “-EncodedCommand” parameter
- Base64 string present in the CommandLine field
- ExecutionPolicy Bypass used simultaneously
- Parent process chain
- User context
- Integrity level

Analysis:

The decoded Base64 string resolved to “whoami,” which is benign. However, the technique itself is commonly associated with malicious activity. Encoded command execution is a known obfuscation method used to:

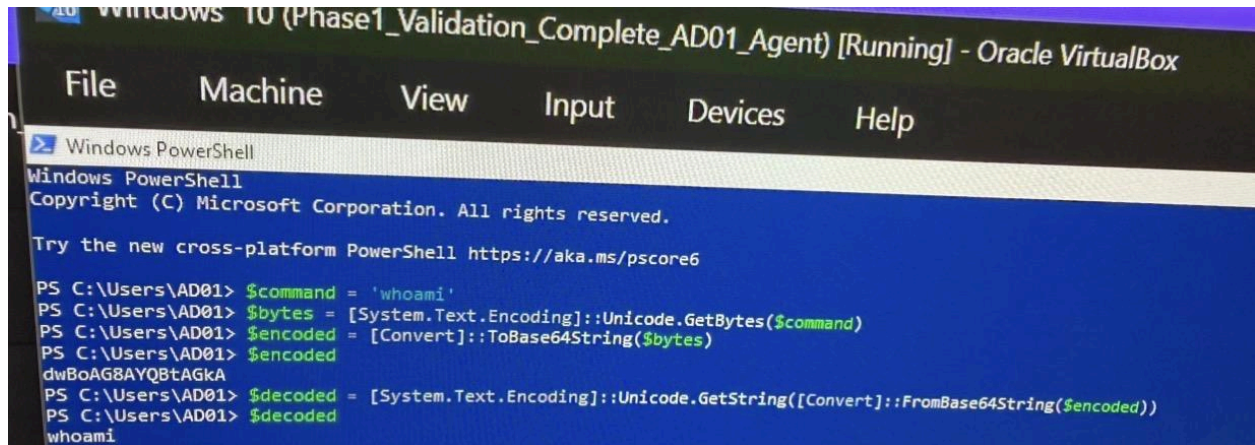
- Conceal payload behavior
- Evade simple string-based detection
- Obfuscate malicious scripts delivered through phishing

Encoded Command Decoding Validation

To simulate investigative workflow, the Base64 string was decoded to confirm its contents.

Evidence:

Decoding result showing the original command (“whoami”).

A screenshot of a Windows PowerShell terminal window. The title bar reads "Windows 10 (Phase1_Validation_Complete_AD01_Agent) [Running] - Oracle VirtualBox". The menu bar includes "File", "Machine", "View", "Input", "Devices", and "Help". The PowerShell prompt shows the user is in the directory "C:\Users\AD01". The user enters a series of commands: first, they set a variable \$command to 'whoami'; then, they convert it to bytes using [System.Text.Encoding]::Unicode.GetBytes(\$command); next, they encode the bytes to a Base64 string using [Convert]::ToBase64String(\$bytes); then, they display the encoded string, which is "dwBoAG8AYQBtAGkA"; finally, they decode the Base64 string back to its original form using [System.Text.Encoding]::Unicode.GetString([Convert]::FromBase64String(\$encoded)), and the output is "whoami".

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Try the new cross-platform PowerShell https://aka.ms/pscore6

PS C:\Users\AD01> $command = 'whoami'
PS C:\Users\AD01> $bytes = [System.Text.Encoding]::Unicode.GetBytes($command)
PS C:\Users\AD01> $encoded = [Convert]::ToBase64String($bytes)
PS C:\Users\AD01> $encoded
dwBoAG8AYQBtAGkA
PS C:\Users\AD01> $decoded = [System.Text.Encoding]::Unicode.GetString([Convert]::FromBase64String($encoded))
PS C:\Users\AD01> $decoded
whoami
```

Screenshot – (Decoded Base64String) Executed on 2026-02-19 11:13 UTC

Analysis:

In a real-world investigation, analysts must decode Base64 strings found in process creation logs to determine actual intent. Detection alone is insufficient; content inspection is required to assess impact and risk.

SOC Investigation Workflow

In a production SOC environment, activity of this nature would require structured triage rather than isolated log review. The following workflow reflects how an analyst would investigate encoded PowerShell execution discovered in endpoint telemetry.

Step 1 – Identify Suspicious PowerShell Execution

An analyst reviewing alerts or performing log analysis would first identify PowerShell process creation events (Sysmon Event ID 1) containing high-risk command-line indicators such as:

- -ExecutionPolicy Bypass
- -EncodedCommand

These parameters significantly elevate risk and warrant deeper inspection.

Step 2 – Extract Critical Telemetry Fields

Once identified, the analyst would extract and review the following fields from the event:

- Image
- ParentImage
- CommandLine

- ParentCommandLine
- User
- IntegrityLevel
- ProcessGuid
- Timestamp

These fields establish execution context and allow reconstruction of process lineage.

Step 3 – Evaluate Parent-Child Process Relationships

The parent process chain is examined to determine how PowerShell was launched. For example:

- Was PowerShell launched directly by a user?
- Was it spawned by a document, script host, or another executable?
- Does the parent process align with normal user behavior?

Process lineage provides critical context when distinguishing malicious activity from legitimate administration.

Step 4 – Extract and Decode Base64 Content

If -EncodedCommand is present, the encoded string is extracted from the CommandLine field and decoded. Decoding is necessary to determine actual command intent, as encoded payloads obscure content during surface-level review.

In this simulation, the decoded command resolved to whoami, which is benign. However, the investigative requirement remains the same regardless of decoded output.

Step 5 – Assess Command Intent

After decoding, the analyst evaluates:

- Whether the command performs reconnaissance, persistence, credential access, or lateral movement
- Whether the behavior aligns with known attacker tradecraft
- Whether the activity aligns with the user's baseline behavior

Even benign commands executed through obfuscated methods may indicate staging activity.

Step 6 – Determine Scope and Impact

The analyst then assesses:

- Whether additional suspicious PowerShell executions occurred
- Whether child processes were spawned
- Whether outbound network connections followed execution
- Whether other endpoints exhibit similar telemetry

This establishes whether the event is isolated or part of a broader compromise attempt.

Step 7 – Classify and Escalate

Based on contextual findings, the activity is categorized appropriately:

- True Positive – Malicious
- True Positive – Suspicious but benign
- False Positive – Legitimate administrative activity

If malicious indicators are confirmed, escalation procedures and containment actions would follow according to incident response playbooks.

Detection Artifacts Identified:

- Sysmon Event ID 1 (Process Creation)
- powershell.exe execution with ExecutionPolicy Bypass
- Use of -EncodedCommand parameter
- Base64-encoded string in CommandLine field
- Parent-child process relationship visibility
- Full command-line telemetry captured in Wazuh

Operational Takeaways:

- Encoded PowerShell activity should be treated as high-risk even when decoded content appears benign.
- ExecutionPolicy Bypass combined with -EncodedCommand significantly increases detection priority.
- Decoding workflow should be standardized in SOC triage procedures.
- Correlation of parent-child process chains increases detection fidelity.

Conclusion

This phase successfully validated that:

- ExecutionPolicy bypass attempts are logged
- Encoded PowerShell commands are visible in endpoint telemetry
- Base64 strings remain recoverable from logs
- The detection pipeline captures sufficient data for investigative analysis

Although the simulated command was benign, the execution method reflects commonly observed attacker techniques in phishing and post-exploitation scenarios. This confirms that the lab environment provides sufficient visibility to detect and investigate obfuscated PowerShell execution.

Security Implications of Encoded PowerShell Execution

ExecutionPolicy Bypass is frequently used by attackers to override local script restrictions. While execution policies are not security boundaries, bypassing them allows scripts to run without administrative configuration changes. In enterprise environments, this behavior is often associated with phishing payloads, loader scripts, and initial-stage malware.

The -EncodedCommand argument significantly increases risk. It allows attackers to hide the true command within a Base64-encoded string, making the activity less obvious in plain-text log review. Many commodity malware frameworks, red team toolkits, and threat actors rely on encoded PowerShell commands to evade detection.

In this simulation, decoding the Base64 string revealed the command whoami, demonstrating how defenders must inspect encoded payloads rather than rely solely on surface-level telemetry.

Defensive Recommendations

While detection of encoded PowerShell execution is critical, preventative controls can significantly reduce exposure to this class of tradecraft.

The following defensive measures strengthen resilience against phishing-delivered PowerShell payloads:

1. Enable PowerShell Script Block Logging

Script Block Logging captures the full content of executed PowerShell scripts, including deobfuscated content at runtime. This provides deeper visibility beyond command-line arguments alone.

2. Enable PowerShell Module Logging

Module logging improves audit depth by recording cmdlet-level execution, allowing analysts to identify suspicious command sequences even when encoded.

3. Restrict ExecutionPolicy Changes via Group Policy

Although execution policies are not security boundaries, enforcing consistent policy settings through Group Policy reduces casual bypass attempts and increases detection signal fidelity.

4. Implement Constrained Language Mode

Constrained Language Mode restricts PowerShell functionality for non-administrative users, limiting abuse potential.

5. Alert on PowerShell Spawning Child Processes

PowerShell launching cmd.exe, wscript.exe, or other system utilities is a high-value behavioral signal and should be monitored.

6. Apply Least Privilege Principles

Reducing local administrative privileges limits the potential impact of successful phishing payload execution.

Detection is most effective when paired with layered preventative controls. Telemetry alone does not stop execution; it enables response.

Detection Engineering Considerations

From a defensive standpoint, the following telemetry fields are critical:

- Image
- ParentImage
- CommandLine
- ParentCommandLine
- ProcessGuid
- User
- IntegrityLevel

Example detection logic patterns include:

Rule 1: Flag PowerShell executions containing
-ExecutionPolicy Bypass

Rule 2: Flag PowerShell executions containing -EncodedCommand

Rule 3: Flag PowerShell spawning cmd.exe with enumeration commands such as whoami or hostname

Rule 4: Correlate parent-child relationships involving PowerShell as a parent process

Example detection logic conceptually:

If Image = powershell.exe
AND CommandLine contains "-EncodedCommand"
THEN trigger High Severity alert

These conditions reflect high-fidelity signals frequently used in real-world detection engineering pipelines.

Detection Gaps and False Positive Considerations

While PowerShell executions containing -ExecutionPolicy Bypass and -EncodedCommand are strong behavioral indicators, they are not exclusively malicious.

Legitimate administrative scripts, enterprise management tools, and automation frameworks may use encoded PowerShell commands for operational purposes. Blindly flagging all encoded PowerShell activity as high severity may generate excessive false positives.

The following considerations are critical when tuning detection logic:

1. Administrative Context Matters

Activity executed by known IT administrators during maintenance windows may represent expected behavior.

2. Baseline User Behavior Is Critical

Encoded PowerShell execution by a finance or HR user carries different risk weight than the same behavior from a systems administrator.

3. Frequency and Pattern Analysis

Repeated encoded executions across multiple endpoints may indicate automated deployment or malicious propagation.

4. Parent Process Context

PowerShell spawned by Microsoft Office applications, browser processes, or unknown executables is significantly more suspicious than interactive administrative usage.

5. Command Content Evaluation

Decoded payload inspection is essential. Benign administrative commands differ substantially from credential dumping, download cradles, or lateral movement behavior.

Effective detection engineering balances sensitivity and precision. High-fidelity alerts require contextual enrichment, not just keyword matching.

Severity Justification

PowerShell executions containing both ExecutionPolicy Bypass and -EncodedCommand represent high-risk behavior due to their frequent association with phishing payloads, loader

frameworks, and post-exploitation tradecraft. While legitimate administrative activity may use these parameters, their presence warrants elevated scrutiny and contextual investigation.

Expected Alert Characteristics

A well-designed alert generated from this activity should include:

Alert Name: Suspicious Encoded PowerShell Execution

Severity: High

MITRE Technique: T1059.001 – Command and Scripting Interpreter: PowerShell

Host: AD01

User: AD01 user context

CommandLine: Full PowerShell execution string

Parent Process: powershell.exe

Decoded Command: whoami

Timestamp: UTC

Process GUID: Unique identifier

The alert should clearly communicate:

- What executed
- How it executed
- Under what user context
- Why it is suspicious
- Associated MITRE technique

This ensures analysts can triage quickly without manually reconstructing process chains.

APPENDIX A – COMMANDS USED IN SIMULATION

The following commands were executed during logging validation, phishing simulation, and encoded PowerShell testing.

Logging Validation Phase

Verify Wazuh agent status

```
sudo /var/ossec/bin/agent_control -l
```

Verify Wazuh manager service

```
sudo systemctl status wazuh-manager
```

Verify ingestion port listening

```
sudo ss -tulnp | grep 1514
```

Monitor Wazuh archive logs

```
sudo tail -f /var/ossec/logs/archives/archives.log
```

Generate manual Windows test event

```
eventcreate /ID 1000 /L APPLICATION /T INFORMATION /SO TestSource /D "Manual test event"
```

Validate Sysmon process creation logging

```
whoami
```

Simulated phishing payload execution

```
powershell -ExecutionPolicy Bypass -File .\Invoice_Update.ps1
```

Encoded Command Simulation

Base64 encoding process

```
$command = 'whoami'
```

```
$bytes = [System.Text.Encoding]::Unicode.GetBytes($command)
```

```
$encoded = [Convert]::ToBase64String($bytes)
```

Execute encoded command

```
powershell -ExecutionPolicy Bypass -EncodedCommand <Base64String>
```

Decode Base64 string

```
$decoded =
```

```
[System.Text.Encoding]::Unicode.GetString([Convert]::FromBase64String($encoded))
```